BASES DE DATOS II

Apuntes Completos para el Examen

Temas 4 y 5

Basados en: Silberschatz, Korth & Sudarshan — Database System Concepts, 6ª ed.

TEMA 4: CONTROL DE CONCURRENCIA

4.1 Introducción: ¿Por qué necesitamos control de concurrencia?

En el Tema 3 vimos que las transacciones deben ser aisladas entre sí. El CONTROL DE CONCURRENCIA es el mecanismo que hace realidad esa propiedad de aislamiento cuando múltiples transacciones se ejecutan simultáneamente.

Sin control de concurrencia, transacciones que se ejecutan a la vez pueden interferir entre sí y producir resultados incorrectos. Los dos mecanismos más usados en la práctica son:

- Protocolos basados en bloqueos (Locking): Las transacciones 'bloquean' los datos antes de acceder. Si el dato está bloqueado de forma incompatible, esperan.
- Aislamiento de instantáneas (Snapshot Isolation): Cada transacción trabaja sobre una 'fotografía' de la base de datos en el momento en que empezó.

4.2 Protocolos Basados en Bloqueos

4.2.1 Los dos modos de bloqueo

Existen dos tipos de bloqueos para controlar el acceso concurrente:

Modo	Símbolo	Permite	Compatible con
Compartido (Shared)	S o C	Leer el dato Q, NO modificarlo	Solo con otros bloqueos Compartidos
Exclusivo (Exclusive)	X	Leer Y modificar el dato Q	NINGÚN otro bloqueo (ni S ni X)

La FUNCIÓN DE COMPATIBILIDAD define qué bloqueos pueden coexistir:

Solicita / Tiene	S (Compartido)	X (Exclusivo)
S (Compartido)	✅ COMPATIBLE	❌ INCOMPATIBLE
X (Exclusivo)	❌ INCOMPATIBLE	❌ INCOMPATIBLE

Interpretación: Si un dato ya tiene bloqueo S, pueden añadirse más S (múltiples lectores simultáneos). Si un dato tiene bloqueo X, nadie más puede acceder (ni leer ni escribir) hasta que se libere.

4.2.2 El Protocolo de Bloqueo en Dos Fases (2PL)

Este protocolo es LA SOLUCIÓN estándar para garantizar la serializabilidad. Divide el ciclo de vida de los bloqueos en dos fases:

Fase	Qué puede hacer	Qué NO puede hacer
Fase de CRECIMIENTO (Growing Phase)	ADQUIRIR bloqueos (S o X)	LIBERAR bloqueos
Fase de DECRECIMIENTO (Shrinking Phase)	LIBERAR bloqueos	ADQUIRIR bloqueos nuevos

El PUNTO DE BLOQUEO es el momento en que la transacción obtiene su ÚLTIMO bloqueo (final de la fase de crecimiento). El orden de los puntos de bloqueo de todas las transacciones determina el orden de serialización.

TEOREMA DEL 2PL
El Protocolo de Bloqueo en Dos Fases GARANTIZA que toda planificación generada es serializable en cuanto a conflictos. Si todas las transacciones cumplen 2PL, el sistema siempre producirá resultados correctos.

PROBLEMA del 2PL básico: puede producir retrocesos en cascada (si T1 libera un bloqueo antes de hacer commit y T2 lee ese dato, si T1 falla hay que abortar T2 también).

4.2.3 Variantes del 2PL

Variante	Regla adicional	Ventaja
2PL Estricto (Strict 2PL)	Los bloqueos EXCLUSIVOS (X) se mantienen hasta el COMMIT/ABORT	Evita retrocesos en cascada (nadie lee datos no confirmados)
2PL Riguroso (Rigorous 2PL)	TODOS los bloqueos (S y X) se mantienen hasta el COMMIT/ABORT	Serialización por orden de commit; más simple de implementar

En la práctica, la mayoría de los SGBD comerciales usan 2PL ESTRICTO o RIGUROSO porque evitan el costoso retroceso en cascada. Los bloqueos se liberan todos juntos al final.

4.2.4 Conversión de bloqueos

Para aumentar la concurrencia, una transacción puede convertir un bloqueo S en X (subir/upgrade) o un X en S (bajar/downgrade):

- SUBIR (Upgrade S→X): Solo en la FASE DE CRECIMIENTO. La transacción que tenía S sobre Q la convierte en X cuando va a escribir. Esto puede causar esperas si otra transacción también tiene S sobre Q.
- BAJAR (Downgrade X→S): Solo en la FASE DE DECRECIMIENTO. Libera la exclusividad cuando ya no se va a escribir más, permitiendo que otros lean.

4.2.5 Prevención de la Inanición (Starvation)

La inanición ocurre cuando una transacción espera indefinidamente porque siempre llegan otras que se interponen. Para evitarla, los bloqueos se conceden con esta regla:

Se concede el bloqueo de modo M sobre Q a Ti si y solo si:

1. No hay otro bloqueo incompatible con M sobre Q
2. No hay NINGUNA otra transacción esperando un bloqueo sobre Q que lo haya pedido ANTES que Ti

Esto garantiza que las peticiones se atienden en orden FIFO y ninguna espera indefinidamente.

4.2.6 Como funciona el 2PL en la practica — ejemplo completo del libro

El siguiente ejemplo es el que aparece en las diapositivas del profesor. T1 transfiere 50 de B a A; T2 calcula un porcentaje de A a B. Muestra exactamente como se intercalan los bloqueos:

```
  T1                T2            Gestor de Concurrencia
  ____________________________________________________
  bloquear-x(B)                    conceder-x(B, T1)
  leer(B)
  B := B - 50
  escribir(B)
                    bloquear-c(A)  conceder-c(A, T2)  [C es compatible con nada de
T1 sobre A]
                    leer(A)
                    bloquear-c(B)  [B tiene X de T1] -> conceder-c(B, T2) ESPERA
  bloquear-x(A)                    conceder-x(A, T1)  [T2 tiene solo C sobre A ->
incompatible!]
                                   --> T1 TAMBIEN ESPERA... INTERBLOQUEO

  Solucion: el gestor detecta el ciclo T1->T2->T1, aborta T2.
  T1 continua, obtiene X(A), termina su transferencia.
  T2 se reinicia mas tarde.

  -------------------------------------------------------
  VARIANTE SIN INTERBLOQUEO (ejemplo correcto de 2PL):
  -------------------------------------------------------
  T1                T2            Gestor
  bloquear-x(B)                    conceder-x(B, T1)
  leer(B)
  B := B - 50
  escribir(B)
                    bloquear-c(A)  conceder-c(A, T2)
                    leer(A)
                    bloquear-c(B)  [T1 tiene X sobre B] -> T2 ESPERA
  bloquear-x(A)                    conceder-x(A, T1)  [nadie tiene X sobre A]
  leer(A)
  A := A + 50
  escribir(A)
  desbloquear(B)                   [T1 libera B -> T2 puede continuar]
                                   conceder-c(B, T2)
                    leer(B)        [T2 lee el valor YA modificado por T1]
  desbloquear(A)
                    desbloquear(A)
                    bloquear-x(B)  conceder-x(B, T2)
                    B:=B+A*0.1
                    escribir(B)
                    desbloquear(B)
```

```
Resultado final correcto: A y B tienen los valores esperados.
```

OBSERVACION CLAVE DEL EJEMPLO
T2 espera porque pide C(B) y T1 ya tiene X(B). Esta espera es CORRECTA: garantiza que T2 lee el B ya modificado por T1. Sin el 2PL, T2 podria leer el valor antiguo de B y el resultado seria incorrecto. Aqui se ve claramente la utilidad del protocolo.

4.3 Tratamiento de Interbloqueos (Deadlocks)

Un INTERBLOQUEO (deadlock) ocurre cuando dos o más transacciones se esperan mutuamente de forma circular:

```
T1 tiene bloqueo X sobre B y solicita X sobre A
T2 tiene bloqueo X sobre A y solicita X sobre B
→ T1 espera a T2, T2 espera a T1 → NINGUNA PUEDE AVANZAR
```

LOS INTERBLOQUEOS SON INEVITABLES CON BLOQUEOS
Los interbloqueos son el precio a pagar por usar bloqueos para garantizar consistencia. Son preferibles a los estados inconsistentes porque se pueden resolver (abortando una transacción), mientras que la inconsistencia en los datos no tiene arreglo automático.

4.3.1 Prevención de Interbloqueos

Existen tres estrategias de prevención:

Estrategia 1: Adquirir todos los bloqueos al principio

Cada transacción adquiere TODOS los bloqueos que necesitará antes de empezar a ejecutar, de forma atómica. Si no puede obtenerlos todos, no toma ninguno.

- Ventaja: Imposible el deadlock (nadie espera bloqueos que ya tiene otro)
- Desventaja: Muy difícil saber de antemano qué bloqueos se necesitarán. Baja utilización (datos bloqueados sin usar).

Estrategia 2: Ordenación de los datos

Se establece un orden total de todos los elementos de datos. Las transacciones solo pueden solicitar bloqueos en ese orden específico. Nunca habrá ciclos porque todos van en la misma dirección.

- El protocolo de árbol es un ejemplo de este enfoque (ordena según jerarquía del árbol)

Estrategia 3: Marcas de tiempo (Esperar-Morir / Herir-Esperar)

Cada transacción recibe una marca de tiempo única al iniciarse. Cuando Ti solicita un bloqueo que tiene Tj, hay dos políticas:

Política	Si Ti más vieja que Tj	Si Ti más joven que Tj	Riesgo
Esperar-Morir (Sin expropiación)	Ti ESPERA (la vieja espera a la joven)	Ti RETROCEDE (muere la joven)	Transacciones jóvenes mueren repetidamente
Herir-Esperar (Con expropiación)	Ti HIERE a Tj (Tj retrocede, Ti avanza)	Ti ESPERA (la joven espera a la vieja)	Transacciones viejas nunca esperan

En ambas políticas, cuando una transacción retrocede, CONSERVA su marca de tiempo original al reiniciarse. Esto evita la inanición: eventualmente la transacción será la más vieja y nunca volverá a retroceder.

Estrategia 4: Timeout

Cada transacción espera como máximo un tiempo T predefinido. Si el bloqueo no se concede en ese tiempo, la transacción se aborta y reinicia. Simple de implementar pero difícil calibrar el timeout adecuado.

4.3.2 Detección y Recuperación de Interbloqueos

En lugar de prevenir, se permite que ocurran interbloqueos y se detectan periódicamente. Se usa el GRAFO DE ESPERA (Wait-for Graph):

- Nodos: transacciones activas
- Aristas: Ti → Tj si Ti está esperando un bloqueo que tiene Tj
- Hay interbloqueo si y solo si el grafo tiene un CICLO

```
Ejemplo de ciclo (deadlock):
T1 → T2 → T3 → T1
T1 espera a T2, T2 espera a T3, T3 espera a T1
→ DEADLOCK: ninguna puede avanzar
```

Cuando se detecta un ciclo, hay que RECUPERARSE eligiendo una o más transacciones 'víctima' para abortar y romper el ciclo:

3. SELECCIÓN DE VÍCTIMA: Elegir la que tenga menor 'coste': tiempo que lleva ejecutando, datos que usa, cuánto trabajo falta, número de retrocesos previos (para evitar inanición).
4. RETROCESO: Puede ser total (abortar y reiniciar desde cero) o PARCIAL (solo retroceder hasta el punto donde adquirió el bloqueo que causa el deadlock, si el sistema guarda suficiente información).
5. PREVENCIÓN DE INANICIÓN: Si una transacción es víctima repetidamente, incluir el número de veces que ha retrocedido como factor de coste para que eventualmente sea elegida menos.

EJERCICIO 2 del Profesor: Prevención con Esperar-Morir

T14 (MT=5), T15 (MT=10), T16 (MT=15). T14 solicita un dato que tiene T15.
¿Qué ocurre? T15 solicita un dato que tiene T14. ¿Qué ocurre?
T16 solicita un dato que tiene T15. ¿Qué ocurre?

SOLUCIÓN:

Con Esperar-Morir: una transacción espera solo si es MÁS VIEJA que quien tiene el bloqueo. Si es más joven, muere (retrocede).

• T14 (MT=5) solicita dato de T15 (MT=10):
  T14 es MÁS VIEJA (MT más pequeño = nació antes). → T14 ESPERA

• T15 (MT=10) solicita dato de T14 (MT=5):
  T15 es MÁS JOVEN que T14. → T15 MUERE (retrocede)

• T16 (MT=15) solicita dato de T15 (MT=10):
  T16 es MÁS JOVEN que T15. → T16 MUERE (retrocede)

NOTA: Las transacciones que retroceden conservan su MT original al reiniciarse.
Así eventualmente serán las más viejas y no volverán a morir → no hay inanición.

4.4 Protocolos Basados en Validación (Control Optimista)

¿Cuándo usar validación?

Los protocolos de bloqueo son PESIMISTAS: asumen que puede haber conflictos y los previenen bloqueando. Si hay pocas transacciones de escritura y muchos conflictos son raros, este pesimismo genera overhead innecesario.

El control OPTIMISTA asume que los conflictos son raros y deja que las transacciones ejecuten libremente. Solo al final, en una fase de validación, comprueba si hubo conflicto. Si lo hubo, aborta.

4.4.1 Las tres fases del protocolo de validación

Fase	Qué hace	En la BD real
1. Fase de LECTURA (Read Phase)	Ejecuta todo: lecturas de BD, cálculos. Las ESCRITURAS se hacen en variables LOCALES temporales, NO en la BD real.	Solo lecturas reales
2. Fase de VALIDACIÓN (Validation Phase)	Realiza la prueba de validación para comprobar que no hay conflicto de serializabilidad.	Ningún cambio aún
3. Fase de ESCRITURA (Write Phase)	Si pasó la validación, copia los valores locales a la BD real. Si no pasó, aborta.	Solo si validación OK

4.4.2 Marcas de tiempo para validación

A cada transacción Ti se le asignan tres marcas de tiempo:

- Start(Ti): Cuando Ti comenzó su ejecución
- Validation(Ti): Cuando Ti terminó la fase de lectura e inicia la validación
- Finish(Ti): Cuando Ti terminó su fase de escritura

El orden de serialización se determina por Validation(Ti). Es decir: TS(Ti) = Validation(Ti).

4.4.3 La prueba de validación

Para que Ti pase la validación, para TODA transacción Tk con TS(Tk) < TS(Ti) debe cumplirse UNA de estas condiciones:

Condición	Significado	¿Por qué es seguro?
Finish(Tk) < Start(Ti)	Tk terminó COMPLETAMENTE antes de que Ti empezara	No hay solapamiento posible → orden serial garantizado
(1) El conjunto de datos escritos por Tk NO interseca con los leídos por Ti (2) Tk termina escritura antes de que Ti empiece validación: Start(Ti) < Finish(Tk) < Validation(Ti)	Tk no afectó lo que leyó Ti, y sus fases de escritura no se solapan	Ti leyó datos consistentes y no hay escritura concurrente

Si ninguna de las dos condiciones se cumple para alguna Tk, Ti ABORTA.

VENTAJA CLAVE
El control optimista evita los retrocesos en cascada automáticamente: las escrituras reales solo ocurren después de la validación (momento del commit). Nunca se leen datos no confirmados porque las escrituras de transacciones en vuelo son invisibles a los demás.

EJERCICIO 2 del Profesor: Protocolo de Validación

Sistema con protocolo de validación. Transacciones con marcas de tiempo:
T1: Inicio=2, Validación=4, Fin=6
T2: Inicio=1, Validación=7, Fin=9
T1 lee A y B, escribe A.
T2 lee A y B, escribe B.
a) ¿Pasan T1 y T2 la validación? b) ¿Hay que abortar alguna?

SOLUCIÓN:
ORDEN DE SERIALIZACIÓN: TS(T1)=Validation(T1)=4, TS(T2)=Validation(T2)=7
→ T1 va antes que T2 en el orden de serialización.

VALIDACIÓN DE T1 (la primera en validar):
No hay Tk con TS(Tk) < TS(T1)=4 que haya completado → condición trivialmente cumplida.
T1 PASA la validación. ✅

VALIDACIÓN DE T2 (TS=7). Tk candidato: T1 (TS=4 < 7).
Condición 1: Finish(T1)=6 vs Start(T2)=1 → Finish(T1)=6 > Start(T2)=1 → NO cumple cond.1.
Condición 2: (a) escritos por T1={A}, leídos por T2={A,B} → A ∈ intersección → ¡CONFLICTO!
T2 NO pasa la validación. ❌ T2 debe ABORTAR.

RAZÓN: T2 leyó A, y T1 escribió A mientras T2 estaba activa. Si se permitiera el commit de T2, el orden sería inconsistente (T2 leyó el A ANTIGUO pero en la serialización T1 va primero).

4.5 Aislamiento de Instantáneas (Snapshot Isolation)

¿Qué es?

El aislamiento de instantáneas es uno de los mecanismos más populares en sistemas reales (Oracle, PostgreSQL, SQL Server). La idea conceptual es sencilla: cuando una transacción Ti empieza, recibe una 'fotografía' (snapshot) de la base de datos en ese instante.

- Ti trabaja sobre esa fotografía: ve todos los datos confirmados en el momento de su inicio, y NADA de lo que hagan otras transacciones mientras Ti está activa.
- Las actualizaciones de Ti se guardan en un espacio de trabajo privado hasta que Ti hace commit.
- Las transacciones de solo lectura NUNCA tienen que esperar y NUNCA se abortan (siempre ven datos consistentes de su snapshot).

4.5.1 Prevención de actualizaciones perdidas

El principal peligro en snapshot isolation es la ACTUALIZACIÓN PERDIDA: dos transacciones concurrentes modifican el mismo dato; la primera lo actualiza, la segunda lo sobreescribe sin haberlo visto.

Hay dos variantes para manejarlo:

Primer Compromiso Gana (First Committer Wins)

Cuando Ti intenta hacer commit, se verifica ATÓMICAMENTE si alguna transacción concurrente ya confirmó una actualización del mismo dato que Ti quiere escribir. Si es así, Ti aborta. La primera en confirmar gana.

Primera Actualización Gana (First Updater Wins)

Se usa un mecanismo de bloqueo de escritura. Cuando Ti intenta actualizar un dato:

6. Si el dato fue actualizado por una transacción concurrente ya confirmada → Ti aborta
7. Si el bloqueo lo tiene Tj (activa), Ti espera a que Tj termine. Si Tj confirma → Ti aborta. Si Tj aborta → Ti puede continuar.

4.5.2 El gran problema: NO garantiza serializabilidad

Snapshot isolation es eficiente pero tiene un defecto importante: NO garantiza serializabilidad completa. Esto ocurre aunque Oracle lo llame 'serializable'. El tipo de anomalía más común es el ATASCO DE ESCRITURA (Write Skew):

```
EJEMPLO DE WRITE SKEW:
```

```
Restricción: CC + Ahorro >= 0 (la suma de dos cuentas no puede ser negativa)
CC = 100, Ahorro = 200 (suma = 300 ✅)

T1 lee CC=100, Ahorro=200, comprueba restricción: 100-200+200=100 >= 0 ✅ →
escribe CC=CC-200=-100
T2 lee CC=100, Ahorro=200, comprueba restricción: 100+200-200=100 >= 0 ✅ →
escribe Ahorro=Ahorro-200=0

Ambas T1 y T2 escriben datos DISTINTOS (CC y Ahorro) → sin conflicto → ambas
hacen commit

Estado final: CC=-100, Ahorro=0 → suma = -100 ❌ VIOLACIÓN DE LA RESTRICCIÓN
(Esto nunca ocurriría en una ejecución serial: si T1 va primero, T2 vería CC=-100
y no pasaría la comprobación)
```

Solución: Usar la cláusula FOR UPDATE en las lecturas que luego condicionan escrituras. Esto hace que el sistema trate la lectura como si fuera una escritura para efectos de control de concurrencia:

```
-- T1 (con FOR UPDATE):
SELECT CC, Ahorro FROM cuentas WHERE id = 1 FOR UPDATE;
-- Esto adquiere un bloqueo de escritura sobre los datos leídos
-- Ahora T2 tendrá un conflicto real y una de las dos abortará
```

4.5.3 Resumen del Snapshot Isolation

Aspecto	Comportamiento
Lecturas	Nunca esperan, nunca se abortan. Ven el snapshot del inicio.
Escrituras	Se validan al commit (primer o primera actualización gana)
Rendimiento	Muy bueno: pocas esperas, pocas aborturas
Serializabilidad	NO garantizada (posible write skew y otras anomalías)
Solución write skew	Usar SELECT ... FOR UPDATE en lecturas que condicionan escrituras
Usado en	Oracle (su 'SERIALIZABLE'), PostgreSQL (antes de v9.1), SQL Server (nivel 'SNAPSHOT')

4.6 Operaciones de Inserción, Borrado y Bloqueos de Índice

4.6.1 Inserción y borrado

Las operaciones INSERT y DELETE se tratan como escrituras para el control de concurrencia:

- DELETE(Q): Necesita bloqueo X sobre Q. Entra en conflicto con leer(Q), escribir(Q), otro borrar(Q) y con insertar(Q) (dependiendo del estado previo de Q).
- INSERT(Q): Se trata como escribir(Q). En 2PL, la transacción obtiene automáticamente bloqueo X sobre el nuevo elemento insertado.

4.6.2 El Fenómeno Fantasma y los Bloqueos de Índice

Ya visto en el Tema 3: una transacción lee un conjunto de filas con cierta condición WHERE, mientras otra inserta una fila que cumple esa condición. Si el control de concurrencia se hace solo a nivel de tupla, este conflicto pasa desapercibido.

Solución — Protocolo de Bloqueo de Índice: En lugar de bloquear solo las tuplas, también se bloquean los NODOS HOJA del árbol B+ que se usaron para encontrar esas tuplas.

Operación	Acción sobre índice B+
Búsqueda (SELECT con predicado)	Bloqueo S sobre todos los nodos hoja accedidos
Inserción, actualización, borrado	Bloqueo X sobre los nodos hoja afectados por la inserción/borrado del valor de la clave en el índice

Así, cuando T1 busca todos los instructores de Física (lee los nodos hoja de ese rango del índice), y T2 intenta insertar un nuevo instructor de Física (necesita bloqueo X sobre esos mismos nodos), habrá un CONFLICTO REAL DE BLOQUEO que el protocolo gestiona correctamente.

> ⚠ **LIMITACIÓN**
> Muchos SGBD comerciales (Oracle, versiones antiguas de PostgreSQL) NO implementan bloqueos de índice por defecto. Esto los hace vulnerables al fenómeno fantasma incluso con nivel de aislamiento SERIALIZABLE. Es un detalle importante para examen.

4.7 Control de Concurrencia en Interacciones con Usuarios

Los protocolos estándar asumen transacciones cortas y automáticas. Pero cuando hay interacción humana (un usuario tarda minutos eligiendo un asiento de avión), los bloqueos son inviables:

Problema	2PL	Validación/Timestamps	Snapshot Isolation
Bloquear durante interacción	Bloquea asientos mientras el usuario decide → inaceptable (¿y si se va sin cancelar?)	El usuario A aborta si B actualiza cualquier asiento, aunque no sea el mismo	Solo aborta si dos usuarios eligen el MISMO asiento (mejor opción)

La solución: dividir y verificar con números de versión

Se divide la transacción en partes que no incluyan la espera del usuario:

8. Transacción 1: Leer disponibilidad de asientos
9. El usuario elige (fuera de cualquier transacción)
10. Transacción 2: Verificar que el asiento sigue disponible Y reservarlo

Para la verificación se usa un NÚMERO DE VERSIÓN en cada fila:

```
-- En el esquema se añade: version_number INT DEFAULT 0

-- Transacción 2 (al confirmar la reserva del usuario):
UPDATE asientos
SET ocupado = true, version_number = version_number + 1
WHERE id = :asiento AND version_number = :version_leida_antes;
-- Si version_number cambió desde que se leyó → alguien modificó el asiento
-- → el UPDATE no afecta ninguna fila → sabemos que hay conflicto → volvemos a
intentar
```

Este esquema se llama CONTROL DE CONCURRENCIA OPTIMISTA SIN VALIDACIÓN DE LECTURA (porque no se validan las lecturas, solo las escrituras). Es la base de frameworks ORM como Hibernate (donde se llaman 'conversaciones' a estas transacciones largas).

🖊 EJERCICIO 3 del Profesor: Snapshot Isolation

Base de datos con A=100, B=200. Snapshot Isolation con 'primer compromiso gana'.
T1: begin, leer(A), leer(B), escribir(A), escribir(B), commit
T2: begin, leer(A), escribir(A), commit
(T1 y T2 empiezan simultáneamente. T1 quiere A:=A+50, B:=B-50. T2 quiere A:=A*2)
a) ¿Qué valores leen T1 y T2?
b) ¿Qué transacciones hacen commit y cuáles abortan?
c) ¿Estado final?

✅ SOLUCIÓN:

a) VALORES LEÍDOS:
T1 lee su snapshot: A=100, B=200
T2 lee su snapshot: A=100
Ambas ven los valores iniciales porque empezaron simultáneamente.

b) COMMIT/ABORT:
T2 es más corta y hace commit primero. Escribe A=200 (100*2). ✅ T2 COMMIT
T1 intenta hacer commit. La validación verifica: ¿alguna transacción concurrente escribió A o B? SÍ: T2 (concurrente) escribió A, y T1 también quiere escribir A.
→ CONFLICTO → T1 ABORTA ❌

c) ESTADO FINAL: A=200 (escrito por T2), B=200 (sin cambios, T1 abortó)

d) NO hay actualización perdida: el sistema detectó el conflicto y abortó T1.

🖊 EJERCICIO 4 del Profesor: Snapshot Isolation Primera Actualización Gana

A=10, B=50. T1: leer A, leer B, A:=A+20, escribir A, B:=B+10, escribir B, commit
T2: leer A, A:=A*3, escribir A, commit. Política: primera actualización gana.
T1 y T2 empiezan simultáneamente. T2 es más rápida.
a) ¿Qué valores leen? b) ¿Qué pasa cuando T1 intenta escribir A? c) ¿Estado final?

✅ SOLUCIÓN:

a) T1 lee su snapshot: A=10, B=50. T2 lee su snapshot: A=10.

b) T2 es más rápida: pide bloqueo de escritura sobre A → lo obtiene → escribe A=30 → commit.
T1 intenta escribir A: pide bloqueo X sobre A.
T2 ya confirmó una actualización de A (está concurrente con T1).
→ T1 ABORTA (la primera que actualizó gana, T2 ya ganó).

c) ESTADO FINAL: A=30 (T2: 10*3=30), B=50 (T1 abortó, no escribió B).

4.8 Resumen del Tema 4

Concepto	Idea clave
2PL	Fase crecimiento (solo adquirir) → Fase decrecimiento (solo liberar). Garantiza serializabilidad.
2PL Estricto	Bloqueos X hasta commit. Evita cascada.
2PL Riguroso	Todos los bloqueos hasta commit. Serialización por orden de commit.
Deadlock	Espera circular. Resolución: abortar una víctima.
Esperar-Morir	Transacción vieja espera, joven muere. Sin expropiación.
Herir-Esperar	Transacción vieja hiere (expropia), joven espera. Con expropiación.
Grafo de espera	Ti→Tj si Ti espera un dato de Tj. Ciclo = deadlock.
Validación (optimista)	Ejecutar libremente, validar al commit. Bueno si pocos conflictos.
Snapshot Isolation	Cada transacción ve snapshot al inicio. Eficiente, no garantiza serializabilidad.
Write Skew	Dos transacciones leen mismo dato, escriben datos distintos. Anomalía de snapshot.
FOR UPDATE	Solución a write skew: trata lectura como escritura para el control de concurrencia.
Bloqueo de índice	Bloquear nodos hoja del B+ para evitar fenómeno fantasma.
Números de versión	Detección de conflictos en transacciones largas con interacción de usuario.

4.9 Ejemplo Completo de 2PL con la Imagen del Profesor

El libro y el profesor muestran el siguiente ejemplo visual de cómo funciona el 2PL con dos transacciones T1 y T2 accediendo a A y B:

```
ESCENARIO DEL PROFESOR: T1 transfiere 50€ de B a A; T2 suma 10% de A a B
```

```
Datos iniciales: A=1000, B=2000

T1                    T2             Gestor Concurrencia
──────────────────────────────────────────────────────
bloquear-x(B)                        Conceder X(B) a T1
leer(B)                              B=2000 en T1
B:=B-50
escribir(B)                          B=1950 (en buffer)
                      bloquear-c(A)  Conceder C(A) a T2
                      leer(A)        A=1000 en T2
                      bloquear-c(B)  ← B tiene X de T1 → INCOMPATIBLE → T2 ESPERA
bloquear-x(A)                        Conceder X(A) a T1  (T2 no tiene X sobre A)
leer(A)                              A=1000 en T1
A:=A+50
escribir(A)                          A=1050 (en buffer)
[PUNTO DE BLOQUEO de T1 → entra en fase decrecimiento]
desbloquear(B)                       Liberar X(B)
                                     T2 puede continuar → Conceder C(B) a T2
                      leer(B)        B=1950 en T2
desbloquear(A)                       Liberar X(A)
                      desbloquear(A) Liberar C(A)
                      bloquear-x(B)  Conceder X(B) a T2 (nadie más tiene B)
                      B:=B+A*0.1   B:=1950+1000*0.1=2050
                      escribir(B)
                      desbloquear(B)

Resultado: A=1050, B=2050 ✅ Correcto (equivale a T1 luego T2 en serie)
```

> **🔑 OBSERVACIÓN CLAVE**
> En el ejemplo, T2 espera porque pide C(B) y T1 ya tiene X(B). Esta espera ES CORRECTA: garantiza que T2 lee el B ya modificado por T1. Sin el protocolo 2PL, T2 podría leer B=2000 (el valor antiguo) y calcular mal.
>
> Fíjate también: T1 adquiere X(A) ANTES de liberar X(B) → fase de crecimiento. Luego libera en fase de decrecimiento.

4.10 El Interbloqueo Visual del Profesor

El profesor muestra este ejemplo específico de interbloqueo que es muy representativo:

```
INTERBLOQUEO CLÁSICO (imagen del libro/profesor):

T1                              T2
bloquear-x(B)                              ← T1 tiene X sobre B
leer(B)
B:=B-50
escribir(B)
                    bloquear-c(A)          ← T2 tiene C sobre A
                    leer(A)
                    bloquear-c(B)          ← B tiene X(T1) → T2 ESPERA a T1
bloquear-x(A)       ←  A tiene C(T2) → T1 ESPERA a T2
```

```
¡DEADLOCK! T1 espera a T2, T2 espera a T1. Nadie puede avanzar.

GRAFO DE ESPERA:
  T1 ──→ T2 (T1 espera que T2 libere A)
  T2 ──→ T1 (T2 espera que T1 libere B)
  HAY CICLO T1→T2→T1 → INTERBLOQUEO CONFIRMADO

RESOLUCIÓN: El sistema elige víctima (ej. T2) → aborta T2 → T1 obtiene X(A) → T1
continúa
```

4.11 Tabla de planificaciones legales bajo 2PL

El 2PL restringe las planificaciones posibles. No todas las planificaciones serializables son generables por 2PL, y no todas las generadas por 2PL son las únicas serializables:

Conjunto	Contenido
Todas las planificaciones posibles	Cualquier interleaving de las instrucciones
Planificaciones serializables (SCC)	Subconjunto: equivalentes a alguna planificación secuencial
Planificaciones generadas por 2PL	Subconjunto de SCC: SIEMPRE son serializables, pero no todas las SCC son generables por 2PL
Planificaciones sin cascada	Subconjunto de SCC: además garantizan no necesitar rollback en cascada
Planificaciones generadas por 2PL Estricto	Subconjunto de 'sin cascada': generadas por 2PL Estricto

TEMA 5: SISTEMA DE RECUPERACIÓN

5.1 Introducción: ¿Qué garantiza el sistema de recuperación?

El sistema de recuperación es responsable de garantizar las propiedades ACID de atomicidad y durabilidad cuando ocurren fallos. Sus objetivos son:

- ATOMICIDAD: Si una transacción falla a medias, deshacer todos sus cambios (como si nunca hubiera ocurrido)
- DURABILIDAD: Los cambios de transacciones confirmadas persisten incluso si el sistema falla justo después del commit
- ALTA DISPONIBILIDAD: Minimizar el tiempo de inactividad tras un fallo

5.2 Clasificación de los Fallos

Tipo de Fallo	Qué ocurre	Qué se pierde	Cómo se recupera
Fallo de la transacción (error lógico)	La transacción no puede continuar: entrada incorrecta, overflow, datos no encontrados	Nada permanente (solo trabajo en memoria)	Rollback de la transacción con el log
Fallo de la transacción (error del sistema)	Deadlock u otro error del sistema impide continuar	Nada permanente	Rollback y reintentar la transacción
Caída del sistema (System Crash)	Fallo de hardware/software: se pierde la memoria volátil (RAM)	Todo lo que estaba en RAM (buffers no volcados)	Usar el log para rehacer/deshacer
Fallo de disco (Disk Failure)	El disco pierde datos (cabezal golpea el disco, fallo en transferencia)	Bloques de disco corruptos o perdidos	Restaurar volcado + aplicar log

📌 **SUPOSICIÓN DE FALLO-PARADA**
Se asume que cuando hay un error hardware o software, el sistema SE PARA pero los datos en disco no se corrompen. Los sistemas bien diseñados tienen verificaciones internas que abortan el sistema ante el primer error, validando esta suposición.

5.3 Tipos de Almacenamiento

Tipo	Características	Ejemplos	Sobrevive a caída
Volátil	Muy rápido, capacidad limitada	RAM, caché CPU	NO

Tipo	Características	Ejemplos	Sobrevive a caída
No Volátil	Más lento, gran capacidad, puede fallar	SSD, HDD, Flash, cintas	SÍ (salvo fallo físico)
Estable (Stable)	Nunca pierde datos (teórico). En práctica: múltiples copias	RAID espejo, replicación remota	SÍ siempre (en la práctica)

Implementando el almacenamiento estable

Para aproximarse al almacenamiento estable se mantienen DOS copias físicas de cada bloque. La operación de escritura sigue este protocolo:

11. Escribir en el PRIMER bloque físico
12. Esperar a que termine correctamente
13. Escribir en el SEGUNDO bloque físico
14. La escritura solo está completa cuando ambos bloques son consistentes

En recuperación, si un bloque tiene error de checksum, se reemplaza con el otro. Si ambos son correctos pero diferentes (escritura interrumpida), se usa el segundo (se deshace la escritura incompleta).

5.4 El Registro Histórico (Log)

5.4.1 ¿Qué es el log?

El LOG es el componente central del sistema de recuperación. Es una secuencia de registros en almacenamiento estable que documenta TODOS los cambios realizados en la base de datos. Es como la 'caja negra' del avión de la BD.

Tipos de registros en el log:

Tipo de registro	Formato	Cuándo se crea
Inicio de transacción	<Ti start>	Cuando Ti empieza a ejecutar
Actualización	<Ti, Xj, V1, V2>	Antes de cualquier modificación de Xj
Commit	<Ti commit>	Cuando Ti confirma con éxito
Abort	<Ti abort>	Cuando Ti se aborta
Solo-rehacer (compensación)	<Ti, Xj, V2>	Durante el rollback de Ti (no tiene V1)
Punto de revisión	<checkpoint L>	Periódicamente

En el registro de actualización <Ti, Xj, V1, V2>:

- Ti: identificador de la transacción
- Xj: elemento de datos modificado (identificado por su posición en disco)
- V1: valor ANTIGUO (para deshacer/rollback)

- V2: valor NUEVO (para rehacer/redo)

5.4.2 Acceso a los Datos — Bloques Fisicos y Memoria Intermedia

Para entender bien la recuperacion, hay que entender exactamente como el SGBD accede a los datos:

Concepto	Donde esta	Nombre tecnico
Bloques en disco	Almacenamiento NO volatil	Bloques FISICOS (physical blocks)
Bloques en RAM	Memoria intermedia (buffer)	Bloques de MEMORIA INTERMEDIA (buffer blocks)

Cada transaccion Ti tiene un AREA DE TRABAJO PRIVADA donde copia los datos con los que trabaja. Las operaciones entre el area de trabajo y la memoria intermedia son:

```
leer(X):
  1. Si el bloque que contiene X no esta en el buffer: ejecutar entrada(Bx) para
traerlo del disco
  2. Copiar el valor de X del bloque en buffer a la variable local xi de la
transaccion
  (la transaccion trabaja con xi, no directamente con el buffer)

escribir(X):
  1. Si el bloque que contiene X no esta en el buffer: ejecutar entrada(Bx) para
traerlo del disco
  2. Copiar el valor de xi (variable local) al bloque en el BUFFER (no al disco
todavia)

  IMPORTANTE: escribir(X) NO escribe en disco inmediatamente.
  salida(Bx) es la operacion que vuelca el bloque del buffer al disco.
  salida(Bx) ocurre cuando: el gestor de buffer necesita espacio, o el SGBD lo
fuerza.
```

5.4.3 Modificacion Inmediata vs Diferida

Hay dos tecnicas para cuando se hacen efectivos los cambios de una transaccion en la BD real:

Tecnica	Cuando modifica la BD	Ventaja	Requiere en log
MODIFICACION INMEDIATA (Immediate Modification)	Mientras la transaccion esta ACTIVA (antes del commit)	Mas concurrencia: los cambios son visibles antes del commit	V1 (valor antiguo) para poder deshacer si falla
MODIFICACION DIFERIDA (Deferred Modification)	Solo cuando la transaccion se COMPROMETE (al hacer commit)	Nunca hay que deshacer: si falla, no escribio nada	Solo V2 (valor nuevo) para rehacer

```
Con MODIFICACION INMEDIATA (la mas comun en la practica):
```

```
  - La transaccion puede escribir en el buffer ANTES de comprometerse
  - Si la transaccion falla -> hay que DESHACER usando V1 del log (UNDO)
  - Si el sistema falla -> puede que algunos datos ya estaban en disco:
    * Transacciones comprometidas: REHACER con V2 (puede que no todos llegaron al
disco)
    * Transacciones activas: DESHACER con V1 (restaurar valores anteriores)

Con MODIFICACION DIFERIDA:
  - La transaccion escribe en variables locales, NO en el buffer ni en disco
  - Solo al commit: copiar todas las variables locales al buffer/disco
  - Si la transaccion falla -> no hay nada que deshacer (no escribio nada en la
BD)
  - Inconveniente: necesita guardar TODOS los valores modificados en memoria
local
```

RELACION CON EL CONTROL DE CONCURRENCIA
El bloqueo en 2 fases (2PL) funciona bien con MODIFICACION INMEDIATA. El aislamiento de instantaneas y validacion prefieren MODIFICACION DIFERIDA (solo escriben al confirmar). Ambas combinaciones son validas, el SGBD elige segun su diseno interno.

5.4.4 La Regla de Escritura Anticipada (Write-Ahead Logging — WAL)

Esta regla es FUNDAMENTAL: el registro del log DEBE escribirse en almacenamiento estable ANTES de que el dato correspondiente se escriba en la base de datos real.

🔑 **REGLA WAL (Write-Ahead Logging)**
ANTES de escribir el bloque Bx en disco (que contiene el dato X modificado), TODOS los registros del log correspondientes a Bx deben estar ya en almacenamiento estable.

Sin esta regla: si el sistema falla justo después de escribir el dato pero antes del log, no habría forma de deshacer la operación (no sabemos el valor anterior).

Consecuencia directa: El log siempre va POR DELANTE de los datos reales en disco. Por eso se llama 'write-AHEAD' (escritura por adelantado).

5.4.5 ¿Cuándo se considera confirmada una transacción?

Una transacción Ti está CONFIRMADA (committed) exactamente cuando su registro <Ti commit> está escrito en almacenamiento estable. Es la acción atómica definitiva.

NOTA IMPORTANTE: Los datos que modificó Ti NO necesitan estar en disco cuando Ti se confirma. Pueden seguir en el buffer (memoria intermedia). El log garantiza que si el sistema falla, podemos REHACER los cambios de Ti al reiniciar.

5.5 Las dos operaciones de recuperación: UNDO y REDO

Operación	Cuándo se usa	Qué usa del log	Efecto
UNDO (deshacer) Ti	Ti no tiene <commit> ni <abort> en el log (estaba activa al fallar)	El valor V1 (antiguo) de cada registro de Ti	Restaura todos los datos de Ti a sus valores anteriores → como si Ti nunca hubiera existido
REDO (rehacer) Ti	Ti tiene <commit> o <abort> en el log	El valor V2 (nuevo) de cada registro de Ti	Aplica todos los cambios de Ti (puede que no llegaron al disco porque el buffer no se volcó)

¿Por qué rehacer también las transacciones ABORTADAS? Cuando una transacción aborta, el proceso de deshacer genera registros SOLO-REHACER en el log. Si el sistema falla durante ese proceso de deshacer, necesitamos rehacer esos registros solo-rehacer para completar el deshacer original. Esto simplifica el algoritmo de recuperación enormemente.

5.6 El Algoritmo de Recuperación

5.6.1 Retroceso normal de una transacción individual (durante operación normal)

Cuando una transacción Ti debe abortarse durante la operación normal (sin que el sistema haya fallado, por ejemplo por un deadlock o un error lógico):

15. Explorar el log HACIA ATRÁS desde el final, buscando registros de Ti
16. Para cada registro <Ti, Xj, V1, V2> encontrado:
    - Aplicar deshacer: escribir V1 (valor antiguo) en Xj en la BD — restaura el dato a como estaba antes
    - Escribir un registro SOLO-REHACER <Ti, Xj, V1> en el log — documenta que se deshizo este cambio
    - Los registros solo-rehacer son importantes: si el sistema falla DURANTE el retroceso, en la recuperacion se pueden rehacer los deshacer ya aplicados
17. Al encontrar el registro <Ti start>, PARAR la exploracion y escribir <Ti abort> en el log

```
Ejemplo de retroceso de Ti:

  LOG antes del retroceso:
  <Ti start>
  <Ti, A, 1000, 950>    <- Ti cambio A de 1000 a 950
  <Ti, B, 2000, 2050>   <- Ti cambio B de 2000 a 2050
  <Ti, C, 500, 480>     <- Ti cambio C de 500 a 480

  Proceso de retroceso (de atras hacia adelante):
  1. Encontrar <Ti, C, 500, 480> -> DESHACER: escribir C=500. Log: <Ti, C, 500>
(solo-rehacer)
  2. Encontrar <Ti, B, 2000, 2050> -> DESHACER: escribir B=2000. Log: <Ti, B,
2000>
  3. Encontrar <Ti, A, 1000, 950> -> DESHACER: escribir A=1000. Log: <Ti, A,
1000>
  4. Encontrar <Ti start> -> Escribir <Ti abort>. PARAR.
```

```
LOG despues del retroceso:
<Ti start>
<Ti, A, 1000, 950>
<Ti, B, 2000, 2050>
<Ti, C, 500, 480>
<Ti, C, 500>           <- solo-rehacer: registro de la operacion de deshacer
<Ti, B, 2000>          <- solo-rehacer
<Ti, A, 1000>          <- solo-rehacer
<Ti abort>             <- Ti ha terminado (abortada)
```

5.6.2 Recuperación tras caída del sistema (en dos fases)

Al reiniciar el sistema tras una caída, se ejecutan dos fases basadas en el último punto de revisión:

FASE 1: REHACER (Repetir la historia)

Se recorre el log HACIA ADELANTE desde el último <checkpoint L>, rehaciendo TODAS las operaciones sin excepción (incluyendo las transacciones que luego se abortan o las que estaban en proceso de deshacer).

18. Establecer la lista-deshacer = L (transacciones activas en el checkpoint)
19. Para cada registro <Ti, Xj, V1, V2> o <Ti, Xj, V2> (solo-rehacer): escribir V2 en Xj
20. Para cada <Ti start>: añadir Ti a lista-deshacer
21. Para cada <Ti commit> o <Ti abort>: eliminar Ti de lista-deshacer
22. Al final de esta fase, lista-deshacer contiene exactamente las transacciones que estaban incompletas al fallar

FASE 2: DESHACER (Limpiar las incompletas)

Se recorre el log HACIA ATRÁS desde el final, deshaciendo las transacciones en lista-deshacer:

23. Para cada registro de una Ti en lista-deshacer: escribir V1 en el dato y añadir registro solo-rehacer
24. Para cada <Ti start> de una Ti en lista-deshacer: escribir <Ti abort> y eliminar Ti de lista-deshacer
25. Terminar cuando lista-deshacer esté vacía

PRINCIPIO DE 'REPETIR LA HISTORIA'

En la Fase 1 se rehacen incluso las transacciones que luego se deshacerán en la Fase 2. Esto parece ineficiente, pero simplifica enormemente el algoritmo: todas las transacciones se tratan igual en la primera pasada. Solo en la segunda pasada se distingue quién se deshace.

EJERCICIO: Algoritmo de Recuperación Completo

Log (de arriba a abajo = más antiguo a más reciente):
<T0 start>
<T0, A, 1000, 950>
<T0, B, 2000, 2050>

<T0 commit>
<T1 start>
<T1, C, 700, 600>
--- El sistema FALLA aquí (T1 no tiene commit) ---
Aplica el algoritmo de recuperación.

✅ **SOLUCIÓN:**

FASE 1 - REHACER (hacia adelante, desde el inicio del log):
No hay checkpoint, así que procesamos todo el log.
lista-deshacer = {} inicialmente
Registro <T0 start>: añadir T0 → lista-deshacer = {T0}
Registro <T0, A, 1000, 950>: REHACER → escribir A=950
Registro <T0, B, 2000, 2050>: REHACER → escribir B=2050
Registro <T0 commit>: eliminar T0 → lista-deshacer = {}
Registro <T1 start>: añadir T1 → lista-deshacer = {T1}
Registro <T1, C, 700, 600>: REHACER → escribir C=600
Fin de fase 1. lista-deshacer = {T1} (T1 estaba incompleta)

FASE 2 - DESHACER (hacia atrás, deshacer T1):
Registro <T1, C, 700, 600>: DESHACER → escribir C=700 (restaurar V1)
Añadir al log: <T1, C, 700> (solo-rehacer)
Registro <T1 start>: escribir <T1 abort> en log, eliminar T1 de lista-deshacer
lista-deshacer = {} → TERMINAR

ESTADO FINAL: A=950, B=2050 (T0 confirmada, sus cambios persisten)
C=700 (T1 abortada, sus cambios deshecho)

5.7 Puntos de Revisión (Checkpoints)

Por que son necesarios?

Sin puntos de revision, en cada recuperacion habria que procesar el log COMPLETO desde el inicio del tiempo. Si el sistema lleva meses funcionando, esto puede ser millones de registros. Los checkpoints limitan cuanto log hay que procesar.

El proceso de crear un checkpoint

26. Escribir todos los registros del log que están en memoria (buffer del log) a almacenamiento estable
27. Escribir en disco todos los bloques de datos modificados que están en la memoria intermedia (buffer de datos)
28. Escribir en el log el registro <checkpoint L>, donde L es la lista de transacciones activas en ese momento

Mientras se realiza el checkpoint (versión simple), NINGUNA transacción puede hacer actualizaciones. En la práctica se usan 'checkpoints difusos' (fuzzy checkpoints) que permiten que las transacciones sigan durante el proceso.

Recuperación con checkpoints

Al recuperarse, en lugar de procesar el log desde el inicio, solo se procesa desde el ÚLTIMO checkpoint:

- Las transacciones que confirmaron ANTES del checkpoint ya tienen sus datos en disco (se hicieron en el paso 2 del checkpoint). No necesitan REDO.
- Las transacciones activas en el checkpoint (lista L) pueden necesitar UNDO si no confirmaron antes del fallo.
- Las transacciones que empezaron DESPUÉS del checkpoint pueden necesitar REDO o UNDO según si confirmaron o no.

EJEMPLO CON CHECKPOINT
Si el checkpoint tiene L={T67, T69}, significa que en ese momento T67 y T69 estaban activas. T1 a T66 ya habian confirmado y sus datos estan en disco. Cualquier Tx con numero < 67 no necesita procesarse en la recuperacion.

EJERCICIO del Profesor: Algoritmo de recuperacion completo CON CHECKPOINT (del libro)

Log completo (el fallo ocurre en el punto indicado). Este es exactamente el ejemplo del libro:
<T0 start>
<T0, B, 2000, 2050>
<T0 commit>
<T1 start>
<T1, C, 700, 600>
<checkpoint {T1}> <- checkpoint con T1 activa
<T2 start>
<T2, A, 500, 400>
<T0 start> <- T0 empieza de nuevo (segunda ejecucion)
<T0, B, 2000, ---> <- OJO: segun el libro: <T0, B, 2000>
<T0 abort>
--- FALLO DEL SISTEMA AQUI ---
Aplica el algoritmo completo de recuperacion con checkpoint.

SOLUCIÓN:
NOTA PREVIA: La fase de recuperacion siempre empieza desde el ULTIMO checkpoint.
El registro <checkpoint {T1}> indica que T1 estaba activa. T0 ya habia confirmado antes.

FASE 1: REHACER (hacia adelante desde el checkpoint)
lista-deshacer = {T1} <- inicializar con la lista del checkpoint

<T2 start>: añadir T2 -> lista-deshacer = {T1, T2}
<T2, A, 500, 400>: REHACER -> A = 400
<T0 start>: añadir T0 -> lista-deshacer = {T1, T2, T0}
<T0, B, 2000, ...>: REHACER segun el valor del log
<T0 abort>: eliminar T0 -> lista-deshacer = {T1, T2}
(Fin del log sin mas registros)

```
  Al final de Fase 1: lista-deshacer = {T1, T2}
  (T1 y T2 no tienen ni <commit> ni <abort> -> deben deshacerse)

FASE 2: DESHACER (hacia atras, solo afecta a T1 y T2)
  Recorremos el log HACIA ATRAS:
  <T0 abort>: T0 NO esta en lista -> saltar
  <T0, B, ...>: T0 NO esta en lista -> saltar
  <T0 start>: T0 NO esta en lista -> saltar
  <T2, A, 500, 400>: T2 SI esta en lista -> DESHACER A=500
                     Escribir en log: <T2, A, 500>  (solo-rehacer)
  <T2 start>: T2 en lista -> escribir <T2 abort>, quitar T2. Lista = {T1}
  <checkpoint {T1}>: T1 sigue en lista, seguir hacia atras
  <T1, C, 700, 600>: T1 SI esta en lista -> DESHACER C=700
                     Escribir en log: <T1, C, 700>  (solo-rehacer)
  <T1 start>: T1 en lista -> escribir <T1 abort>, quitar T1. Lista = {}
  Lista vacia -> TERMINAR

ESTADO FINAL:
  B = 2050 (T0 confirmo en la primera ejecucion -> su cambio persiste)
  A = 500  (T2 deshecha -> vuelve a su valor original)
  C = 700  (T1 deshecha -> vuelve a su valor original)
```

5.8 Fallo con pérdida de almacenamiento no volátil

Los fallos de disco (pérdida de datos en el disco duro) son más raros pero más graves. La estrategia es el VOLCADO PERIÓDICO (dump/backup):

El proceso de volcado

29. Escribir todos los registros del log de memoria a almacenamiento estable
30. Escribir todos los bloques de datos modificados al disco
31. COPIAR TODO el contenido de la base de datos a almacenamiento estable (cintas, otro disco)
32. Escribir <dump> en el log

Recuperación tras fallo de disco

33. RESTAURAR la BD desde el último volcado
34. Consultar el log y REHACER todas las transacciones confirmadas desde el último volcado
35. No hay que deshacer nada (el volcado ya era un estado consistente, y solo rehacemos las confirmadas)

Tipo de fallo	Qué se usa para recuperar	Requiere UNDO
Caída del sistema (RAM perdida)	Log + último checkpoint	SÍ (transacciones incompletas)
Fallo de disco	Último volcado + Log desde el volcado	NO (solo rehacer confirmadas)

📌 **VOLCADO SQL vs FÍSICO**
Un volcado SQL genera sentencias DDL + INSERT que reconstruyen la BD desde cero. Es más lento de aplicar pero portable entre versiones y sistemas. El volcado físico es más rápido pero específico del SGBD. Los SGBDs modernos ofrecen volcados difusos (fuzzy dumps) que permiten continuar operando durante el volcado.

5.9 Políticas de Gestión del Buffer

El buffer manager decide cuándo escribir los bloques de la memoria intermedia al disco. Hay dos dimensiones independientes:

Política	Descripción	Pros	Contras
Forzar (Force)	Al commit, escribir TODOS los bloques modificados al disco	Recuperación fácil: no hay redo	Muy lento: muchos accesos a disco en cada commit
No-Forzar (No-Force)	Al commit, NO es necesario escribir al disco (los datos pueden quedar en buffer)	Más rápido: se agregan escrituras	Recuperación necesita REDO
Apropiación (Steal)	Permitir que bloques de transacciones activas se escriban al disco antes del commit	Libera memoria buffer	Recuperación necesita UNDO
No-Apropiación (No-Steal)	No escribir al disco bloques de transacciones que no han confirmado	Recuperación no necesita UNDO	Necesita más memoria buffer

En la práctica, la mayoría de los SGBD usan NO-FORZAR + APROPIACIÓN (la combinación más eficiente en rendimiento) con la regla WAL para garantizar la corrección. Esto requiere REDO para la durabilidad y UNDO para la atomicidad.

5.10 Resumen del Tema 5

Concepto	Idea clave
Log (Registro Histórico)	Secuencia de cambios en almacenamiento estable. Base de toda la recuperación.
Regla WAL	El log va ANTES que los datos al disco. Write-Ahead Logging.
Transacción confirmada	Cuando <Ti commit> está en almacenamiento estable.
UNDO	Deshacer transacción incompleta usando V1 (valores anteriores del log)

Concepto	Idea clave
REDO	Rehacer transacción confirmada usando V2 (valores nuevos del log)
Fase 1 recuperación	REHACER hacia adelante desde último checkpoint. Repetir la historia.
Fase 2 recuperación	DESHACER hacia atrás las transacciones incompletas (en lista-deshacer)
Checkpoint	Punto de referencia que limita cuánto log hay que procesar en recuperación
Volcado (Dump)	Copia completa de la BD para recuperarse de fallos de disco
No-Forzar + Apropiación	Política de buffer más eficiente, requiere WAL + log con REDO y UNDO